

MI DVP API

1 Overview

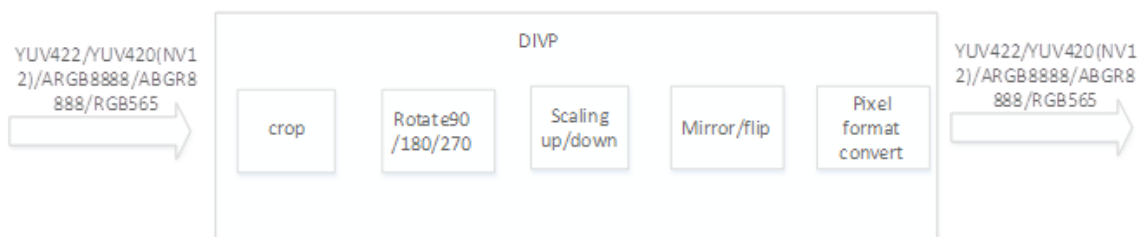
1.1. Module Description

DVP supports preprocessing of an input image, such as cropping, etc., and then scaling each channel separately, and finally outputting images of different resolutions.

The specific image processing functions supported by DVP include:

- cropping of images
 - Image pixel format conversion
 - image rotation
 - Image mirror operation
 - Stretching and scaling of images
-

1.2. Flow Diagram



Notice : Chips that do not support image rotation are listed in **Macaron** 、**Taiyaki** 、**Takoyaki** 、**Ispahan** series.

1.3. Keyword Description

keywords	illustrate
crop	image cropping
rotate	Image rotation
scaling up/down	Image stretching and scaling
Mirror/flip	Image mirror operation
Pixel format convert	Image pixel format conversion

2. API Reference

2.1. Function Module API

API name	Features
MI_DIVP_CreateChn [#22-mi_divp_createchn]	Create a DIVP channel
MI_DIVP_DestroyChn [#23-mi_divp_destroychn]	Destroy a DIVP channel
MI_DIVP_SetChnAttr [#24-mi_divp_setchnattr]	Set the properties of the DIVP channel
MI_DIVP_GetChnAttr [#25-mi_divp_getchnattr]	Get the properties of the DIVP channel
MI_DIVP_StartChn [#26-mi_divp_startchn]	open a channel
MI_DIVP_StopChn [#27-mi_divp_stopchn]	disable a channel
MI_DIVP_SetOutputPortAttr [#28-mi_divp_setoutputportattr]	Set the properties of the DIVP output port
MI_DIVP_GetOutputPortAttr [#29-mi_divp_getoutputportattr]	Get the properties of the DIVP output port
MI_DIVP_RefreshChn [#210-mi_divp_refreshchn]	Refresh a channel in a paused state
MI_DIVP_StretchBuf [#211-mi_divp_stretchbuf]	Scale, stretch, and crop image data in specified memory
MI_DIVP_InitDev [#212-mi_divp_initdev]	Initialize DIVP device
MI_DIVP_DeInitDev [#213-mi_divp_deinitdev]	Deinitialize DIVP device
MI_DIVP_StretchBufEx [#214-mi_divp_stretchbufex]	Scale, stretch, and crop the image data in the specified memory area, and output to the target memory

2.2. MI_DIVP_CreateChn

- Features

Create a new DIVP channel.

- grammar

```
MI_S32 MI_DIVP_CreateChn (
    MI_DIVP_CHN DivpChn,
    MI_DIVP_ChnAttr_t * pstAttr);
```

- 形参

参数名称	描述	输入/输出
DivpChn	返回被创建的DIVP通道的ID，DIVP最多支援6个通道,取值范围[0,5]	输入
pstAttr	DVIP通道属性的指针，用于设定被创建的通道的属性	输入

- 返回值

- MI_SUCCESS 成功创建一个新的DIVP通道。
- MI_DIVP_ERR_FAILED 创建DIVP通道失败
- MI_DIVP_ERR_NO_RESOUCE 系统资源不足，创建DIVP通道失败

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 注意

DIVP 模块最多支援6个channel，当上层创建的channel总数大于6个或者系统资源不足时会创建失败。

- 举例

```
MI_DIVP_CHN u32ChnId = 0;

MI_DIVP_ChnAttr_t stDivpChnAttr;

MI_DIVP_OutputPortAttr_t stDivpOutputPortAttr;

memset(&stDivpChnAttr, 0, sizeof(MI_DIVP_ChnAttr_t));

memset(&stDivpOutputPortAttr, 0, sizeof(MI_DIVP_OutputPortAttr_t));


stDivpChnAttr.bHorMirror = false;

stDivpChnAttr.bVerMirror = false;

stDivpChnAttr.eDiType = E_MI_DIVP_DI_TYPE_OFF;

stDivpChnAttr.eRotateType = E_MI_SYS_ROTATE_90;

stDivpChnAttr.eTnrLevel = E_MI_DIVP_TNR_LEVEL_OFF;

stDivpChnAttr.stCropRect.u16X = 0;

stDivpChnAttr.stCropRect.u16Y = 0;

stDivpChnAttr.stCropRect.u16Width = 1280;

stDivpChnAttr.stCropRect.u16Height = 720;

stDivpChnAttr.u32MaxWidth = 1920;

stDivpChnAttr.u32MaxHeight = 1080;

MI_DIVP_CreateChn(u32ChnId, &stDivpChnAttr);

MI_DIVP_GetChnAttr(u32ChnId, &stDivpChnAttr);

stDivpChnAttr.stCropRect.u16X = 0;

stDivpChnAttr.stCropRect.u16Y = 0;

stDivpChnAttr.stCropRect.u16Width = 1920;

stDivpChnAttr.stCropRect.u16Height = 1080;

MI_DIVP_SetChnAttr(u32ChnId, &stDivpChnAttr);

stDivpOutputPortAttr.eCompMode = E_MI_SYS_COMPRESS_MODE_NONE;
```

```

stDivpOutputPortAttr.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV422_YUYV;

stDivpOutputPortAttr.u32Width = 1920;

stDivpOutputPortAttr.u32Height = 1080;

MI_DIVP_SetOutputPortAttr(u32ChnId, &stDivpOutputPortAttr);

MI_DIVP_StartChn(u32ChnId);

//exit flow

MI_DIVP_StopChn(u32ChnId);

MI_DIVP_DestroyChn(u32ChnId);

```

- 相关主题

[MI_DIVP_DestroyChn](#) [#23-mi_divp_destroychn]

2.3. MI_DIVP_DestroyChn

- 功能

销毁一个 DIVP 通道。

- 语法

```
MI_S32 MI_DIVP_DestroyChn(MI_DIVP_CHN DivpChn);
```

- 形参

参数名称	描述	输入/输出
DivpChn	被销毁的DIVP通道的ID	输入

- 返回值

- MI_SUCCESS 成功销毁DIVP通道。

- MI_DIVP_ERR_FAILED 销毁DIVP通道失败。
- 依赖
 - 头文件：mi_divp.h、mi_divp_datatype.h
 - 库文件：libmi_divp.so
- 相关主题

[MI_DIVP_CreateChn](#) [#22-mi_divp_createchn]

2.4. MI_DIVP_SetChnAttr

- 功能
设置 DIVP channel的属性。
- 语法

```
MI_S32 MI_DIVP_SetChnAttr(  
    MI_DIVP_CHN DivpChn,  
    MI_DIVP_ChnAttr_t * pstAttr);
```

- 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入
pstAttr	设定的DIVP通道的属性的结构体指针	输入

- 返回值
 - MI_SUCCESS 成功设定DIVP通道的属性
 - MI_DIVP_ERR_FAILED 设置DIVP通道的属性失败，
MI_ERR_INVALID_PARAMETER 传入的参数有问题，请参考
[MI_DIVP_ChnAttr_t](#) [#mi_divp_chnattr_t]
- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 注意

通道属性结构体成员中输入图像的最大宽度与最大高度为静态属性，通道创建后不能改变。

- 相关主题

[MI_DIVP_GetChnAttr](#) [#25-mi_divp_getchnattr]

2.5. MI_DIVP_GetChnAttr

- 功能

获取 DIVP channel的属性。

- 语法

```
MI_S32 MI_DIVP_GetChnAttr(  
  
MI_DIVP_CHN DivpChn,  
  
MI_DIVP_ChnAttr_t * pstAttr);
```

- 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入
pstAttr	传回的DIVIP通道的属性的结构体指针	输出

- 返回值

- MI_SUCCESS 成功获取DIVP通道的属性。
- MI_DIVP_ERR_FAILED 获取DIVP通道的属性失败。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h

- 库文件：libmi_divp.so

- 注意

第一次获取时返回创建时的初始值。

- 相关主题

[MI_DIVP_SetChnAttr](#) [#24-mi_divp_setchnattr]

2.6. MI_DIVP_StartChn

- 功能

开启一个通道。

- 语法

```
MI_S32 MI_DIVP_StartChn(MI_DIVP_CHN DivpChn);
```

- 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入

- 返回值

- MI_SUCCESS 成功启用DIVP通道。
- MI_DIVP_ERR_FAILED 启用DIVP通道失败。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 注意

通道必须已经被创建，且没有被销毁。

- 相关主题

[MI_DIVP_StopChn](#) [#27-mi_divp_stopchn]

2.7. MI_DIVP_StopChn

- 功能

禁用一个通道。

- 语法

```
MI_S32 MI_DIVP_StopChn(MI_DIVP_CHN DivpChn);
```

- 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入

- 返回值

- MI_SUCCESS 成功禁用DIVP通道。
- MI_DIVP_ERR_FAILED 禁用DIVP通道失败。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 注意

重复禁用一个通道会返回MI_SUCCESS。

- 相关主题

[MI_DIVP_StartChn](#) [#26-mi_divp_startchn]

2.8. MI_DIVP_SetOutputPortAttr

- 功能

DIVP 通道上output port的属性。

- 语法

```
MI_S32 MI_DIVP_SetOutputPortAttr (
    MI_DIVP_CHN DivpChn,
    MI_DIVP_OutputPortAttr_t* pstOutputPortAttr);
```

- 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入
pstOutputPortAttr	Output port的属性的指针	输入

- 返回值

- MI_SUCCESS 成功设定DIVP通道绑定的output port的属性。
- MI_DIVP_ERR_FAILED 设定DIVP通道绑定的output port的属性失败。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 相关主题

[MI_DIVP_GetOutputPortAttr](#) [#29-mi_divp_getoutputportattr]

2.9. MI_DIVP_GetOutputPortAttr

- 功能

获取 DIVP output port的属性。

- 语法

```
MI_S32 MI_DIVP_GetOutputPortAttr(
    MI_DIVP_CHN DivpChn,
    MI_DIVP_OutputPortAttr_t* pstOutputPortAttr);
```

• 形参

参数名称	描述	输入/输出
DivpChn	DIVP 通道的ID	输入
pstOutputPortAttr	Output port的属性的指针	输出

• 返回值

- MI_SUCCESS 成功获取DIVP通道绑定的output port的属性。
- MI_DIVP_ERR_FAILED 获取DIVP通道绑定的output port的属性失败。

• 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

• 相关主题

[MI_DIVP_SetOutputPortAttr](#) [#28-mi_divp_setoutputportattr]

2.10. MI_DIVP_RefreshChn

• 功能

暂停状态下刷新DIVP channel。

• 语法

```
MI_S32 MI_DIVP_RefreshChn (  
  
MI_DIVP_CHN DivpChn);
```

• 形参

参数名称	描述	输入/输出
DivpChn	被刷新的channel的ID	输入

- 返回值
 - MI_SUCCESS 成功刷新DIVP的channel。
 - MI_DIVP_ERR_NO_CONTENT 该通道没有内容。

- 依赖
 - 头文件：mi_divp.h、mi_divp_datatype.h
 - 库文件：libmi_divp.so

- 注意

该接口适用于channel在暂停的状态下重新刷新。

- 举例

DIVP绑定VDEC的场景下，调用该接口，DIVP可以保留VDEC停止送流后的最后一帧画面，并重复处理然后送给后端，直到VDEC重新开始送流，DIVP才会处理新的画面。

2.11. MI_DIVP_StretchBuf

- 功能

缩放、拉伸、裁剪指定内存区域中的图像数据，并输出到目标内存。

- 语法

```
MI_S32 MI_DIVP_StretchBuf(MI_DIVP_DirectBuf_t *pstSrcBuf,  
MI_SYS_WindowRect_t *pstSrcCrop, MI_DIVP_DirectBuf_t  
*pstDstBuf);
```

2.12. MI_DIVP_InitDev

- 描述

初始化divp设备。

- 语法

```
MI_S32 MI_DIVP_InitDev(MI_DIVP_InitParam_t *pstInitParam);
```

- 参数

参数名称	描述	输入/输出
pstInitParam	设备初始化参数	输入

- 返回值
 - MI_OK 成功
 - 非MI_OK 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_divp.h、mi_divp_datatype.h
 - 库文件：libmi_divp.so

2.13. MI_DIVP_DeInitDev

- 描述

反初始divp设备。
- 语法

```
MI_S32 MI_DIVP_DeInitDev(void);
```

- 返回值
 - MI_OK 成功。
 - 非MI_OK 失败，参照[错误码](#) [#jump]。
- 依赖
 - 头文件：mi_divp.h、mi_divp_datatype.h
 - 库文件：libmi_divp.so
- 注意
 - 此函数必须在初始化设备后调用，否则返回失败。
 - 如果本接口在app退出前没有调用，内部会自动反初始化设备。

- 形参

|参数名称|描述|输入/输出| |--|--|--|--| |pstSrcBuf| 用来存放待处理图像内存信息的结构体指针，不可以为NULL| 输入| |pstSrcCrop |图像裁剪相关信息的结构体指针，如果不需要做裁剪，可以传NULL| 输入| |pstDstBuf| 保存处理后图像内存信息的结构体指针，不可以为NULL| 输入|

- 返回值

- MI_SUCCESS 图像处理成功。
- MI_DIVP_ERR_FAILED 图像处理失败。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

- 注意

该接口只支持YUV420SP或者ARGB8888的像素格式。

- 举例

```
#define SRC_WIDTH 1280
#define SRC_HEIGHT 720
#define SRC_BUFF_STRIDE (ALIGN_UP(SRC_WIDTH,16))
#define SRC_BUFF_SIZE (SRC_BUFF_STRIDE*SRC_HEIGHT*3/2)

#define DST_WIDTH 640
#define DST_HEIGHT 480
#define DST_BUFF_STRIDE (ALIGN_UP(DST_WIDTH,16))
#define DST_BUFF_SIZE (DST_BUFF_STRIDE*DST_HEIGHT*3/2)

#define CROP_X 200
#define CROP_Y 100
#define CROP_W 68
#define CROP_H 48

int main(void)
{
    MI_PHY phySrcBufAddr = 0;
    MI_PHY phyDstBufAddr = 0;
    MI_DIVP_DirectBuf_t stSrcBuf;
    MI_DIVP_DirectBuf_t stDstBuf;
    MI_SYS_WindowRect_t stSrcCrop;

    MI_SYS_MMA_Alloc(NULL, SRC_BUFF_SIZE, &phySrcBufAddr);
```

```

MI_SYS_MMA_Alloc(NULL, DST_BUFF_SIZE, &phyDstBufAddr);

    stSrcBuf.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
    stSrcBuf.u32Width = SRC_WIDTH;
    stSrcBuf.u32Height = SRC_HEIGHT;
    stSrcBuf.u32Stride[0] = SRC_BUFF_STRIDE;
    stSrcBuf.u32Stride[1] = SRC_BUFF_STRIDE;
    stSrcBuf.phyAddr[0] = phySrcBufAddr;
    stSrcBuf.phyAddr[1] = stSrcBuf.phyAddr[0] +
SRC_BUFF_STRIDE*SRC_HEIGHT;
    stDstBuf.ePixelFormat =
E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_420;
    stDstBuf.u32Width = DST_WIDTH;
    stDstBuf.u32Height = DST_HEIGHT;
    stDstBuf.u32Stride[0] = DST_BUFF_STRIDE;
    stDstBuf.u32Stride[1] = DST_BUFF_STRIDE;
    stDstBuf.phyAddr[0] = phyDstBufAddr;
    stDstBuf.phyAddr[1] = stDstBuf.phyAddr[0] +
DST_BUFF_STRIDE*DST_HEIGHT;

    stSrcCrop.u16X = CROP_X;
    stSrcCrop.u16Y = CROP_Y;
    stSrcCrop.u16Width = CROP_W;
    stSrcCrop.u16Height = CROP_H;

    if(FillSrcBuf("./1280x720_yuv420.yuv", &stSrcBuf))
        return NULL;
    if(MI_SUCCESS == MI_DIVP_StretchBuf(&stSrcBuf, &stSrcCrop,
&stDstBuf))
    {
        if(DumpDstBuf(&stDstBuf))
            return NULL;
    }

    MI_SYS_MMA_Free(phySrcBufAddr);
    MI_SYS_MMA_Free(phyDstBufAddr);
}

//fill src buff with yuv420/argb8888 image data
static int FillSrcBuf(const char* FilePath,MI_DIVP_DirectBuf_t
*pstDirectSrcBuf)
{
    int ret = 0;
    FILE *fp;
    void *pVirSrcBufAddr = NULL;
    int LineIdx = 0;
    int ReadSize = 0;

```

```

    fp = fopen(FilePath, "r");
    if(!fp)
    {
        divp_ut_dbg("open file[%s] failed\n", FilePath);
        ret = -1;
        goto EXIT;
    }

    MI_SYS_Mmap(pstDirectSrcBuf->phyAddr[0], SRC_BUFF_SIZE,
    &pVirSrcBufAddr, FALSE);
    if(!pVirSrcBufAddr)
    {
        divp_ut_dbg("mmap dst buff failed\n");
        ret = -1;
        goto EXIT;
    }

    for(LineIdx = 0; LineIdx < SRC_HEIGHT*3/2; LineIdx++)
    {
        ReadSize +=
        fread(pVirSrcBufAddr+LineIdx*SRC_BUFF_STRIDE, 1, SRC_WIDTH,
        fp);
    }
    if(ReadSize < SRC_WIDTH*SRC_HEIGHT*3/2)
    {
        fseek(fp, 0, SEEK_SET);
        ReadSize = 0;
        for(LineIdx = 0; LineIdx < SRC_HEIGHT*3/2; LineIdx++)
        {
            ReadSize +=
            fread(pVirSrcBufAddr+LineIdx*SRC_BUFF_STRIDE, 1, SRC_WIDTH,
            fp);
        }
        if(ReadSize < SRC_WIDTH*SRC_HEIGHT*3/2)
        {
            divp_ut_dbg("read file failed, read
size:%d\n", ReadSize);
            ret = -1;
            goto EXIT;
        }
    }

EXIT:
    if(fp)
        fclose(fp);
    if(pVirSrcBufAddr)
        MI_SYS_Munmap(pVirSrcBufAddr, SRC_BUFF_SIZE);

    return ret;

```

```

}

//image processing result is stored in dst buff
static int DumpDstBuf(MI_DIVP_DirectBuf_t *pstDirectDstBuf)
{
    int ret = 0;
    FILE *fp;
    void *pVirDstBufAddr = NULL;
    int LineIdx = 0;
    int WriteSize = 0;
    char outputfile[128];
    struct timeval timestamp;

    gettimeofday(&timestamp, 0);
    sprintf(outputfile,
"output_%dx%d_%d_%08d.yuv",pstDirectDstBuf-
>u32Width,pstDirectDstBuf->u32Height,(int)timestamp.tv_sec,
(int)timestamp.tv_usec);

    fp = fopen(outputfile,"w+");
    if(!fp)
    {
        divp_ut_dbg("open file[%s] failed\n",outputfile);
        ret = -1;
        goto EXIT;
    }

    MI_SYS_Mmap(pstDirectDstBuf->phyAddr[0], DST_BUFF_SIZE,
&pVirDstBufAddr, FALSE);
    if(!pVirDstBufAddr)
    {
        divp_ut_dbg("mmap dst buff failed\n");
        ret = -1;
        goto EXIT;
    }

    for(LineIdx = 0; LineIdx < DST_HEIGHT*3/2; LineIdx++)
    {
        WriteSize +=
fwrite(pVirDstBufAddr+LineIdx*DST_BUFF_STRIDE, 1, DST_WIDTH,
fp);
    }
    if(WriteSize < DST_WIDTH*DST_HEIGHT*3/2)
    {
        divp_ut_dbg("write file failed, write
size:%d\n",WriteSize);
    }
    fflush(fp);
    sync();
}

```

```

divp_ut_dbg("save stretch dst buff to[%s]\n",outputfile);

EXIT:
    if(fp)
        fclose(fp);
    if(pVirDstBufAddr)
        MI_SYS_Munmap(pVirDstBufAddr, DST_BUFF_SIZE);

    return 0;
}

```

2.14. MI_DIVP_StretchBufEx

- 描述

缩放、拉伸、裁剪指定内存区域中的图像数据，并输出到目标内存。

- 语法

```

MI_S32 MI_DIVP_StretchBufEx(MI_DIVP_DirectBuf_t *pstSrcBuf,
MI_SYS_WindowRect_t *pstSrcCrop, MI_DIVP_DirectBuf_t
*pstDstBuf, MI_DIVP_FilterType_e eFilterType);

```

- 参数

参数名称	描述	输入/输出
pstSrcBuf	源buffer信息指针	输入
pstSrcCrop	Crop信息指针	输入
pstDstBuf	目标buffer信息指针	输入
eFilterType	做scaling的filter type。	输入

- 返回值

- MI_OK 成功
- 非MI_OK 失败，参照[错误码](#) [jump]。

- 依赖

- 头文件：mi_divp.h、mi_divp_datatype.h
- 库文件：libmi_divp.so

3. DIVP 数据类型

3.1. 数据类型定义

数据类型	定义
MI_DIVP_DiType_e [#32-mi_divp_ditype_e]	定义 DIVP 的deinterlace的类型
MI_DIVP_TnrLevel_e [#33-mi_divp_tnrlevel_e]	定义 DIVP TNR的等级
MI_DIVP_FilterType_e [#34-mi_divp_filtertype_e]	定义 DIVP scaling filter type
MI_DIVP_OutputPortAttr_t [#35-mi_divp_outputportattr_t]	定义 DIVP 绑定的output port的属性参数
MI_DIVP_ChnAttr_t [#36-mi_divp_chnattr_t]	定义 DIVP 通道的属性参数
MI_DIVP_DirectBuf_t [#37-mi_divp_directbuf_t]	用于图像处理的内存信息结构体
MI_DIVP_InitParam_t [#38-mi_divp_initparam_t]	定义DIVP设备初始化参数

注：本节已涵盖各重要的数据类型，部分未列出数据类型请参见 **mi_divp_datatype.h**

3.2. MI_DIVP_DiType_e

- 说明
定义 DIVP 的deinterlace的类型。
- 定义

```
typedef enum
{
    E_MI_DIVP_DITYPE_OFF, //off

    E_MI_DIVP_DITYPE_2D, ///2.5D DI

    E_MI_DIVP_DITYPE_3D, ///3D DI

    E_MI_DIVP_DITYPE_NUM,

} MI_DIVP_DiType_e;
```

- 成员

成员	描述
E_MI_DIVP_DITYPE_OFF	DIVP通道上关闭deinterlace
E_MI_DIVP_DITYPE_2D	DIVP通道上开启2.5D deinterlace
E_MI_DIVP_DITYPE_3D	DIVP通道上开启3D deinterlace
E_MI_DIVP_DITYPE_NUM	DIVP通道上deinterlace类型个数

- 注意事项

- 开启DI时必须开TNR，MSR930只能支援3D DI，TNR level设定为E_MI_DIVP_TNRLEVEL_MIDDLE。
- 3D DI与rotation冲突，两个功能不能同时打开。
- 不支持DI功能的chip如下：
 - Pretzel
 - Macaron
 - Taiyaki/Takoyaki
 - Pudding
 - Ispahan

- lkayaki

3.3. MI_DIVP_TnrLevel_e

- 说明
定义 DIVP TNR的等级。
- 定义

```
typedef enum
{
    E_MI_DIVP_TNRLEVEL_OFF,
    E_MI_DIVP_TNRLEVEL_LOW,
    E_MI_DIVP_TNRLEVEL_MIDDLE,
    E_MI_DIVP_TNRLEVEL_HIGH,
    E_MI_DIVP_TNRLEVEL_NUM,
} MI_DIVP_TnrLevel_e;
```

- 成员

成员	描述
E_MI_DIVP_TNRLEVEL_OFF	DIVP通道上关闭TNR
E_MI_DIVP_TNRLEVEL_LOW	DIVP通道上开启弱等级的TNR
E_MI_DIVP_TNRLEVEL_MIDDLE	DIVP通道上开启中等等级的TNR
E_MI_DIVP_TNRLEVEL_HIGH	DIVP通道上开启强等级的TNR
E_MI_DIVP_TNRLEVEL_NUM	DIVP通道上TNR等级的数量

- 注意事项

- MSR930支持TNR，但是不支持调节TNR level。
- 不支持TNR功能的chip如下：
 - Pretzel
 - Macaron
 - Taiyaki/Takoyaki
 - Pudding
 - Ispahan
 - Ikayaki

3.4. MI_DIVP_FilterType_e

- 说明
定义 DIVP scaling filter type。
- 定义

```
typedef enum
{
    E_MI_DIVP_FILTER_TYPE_AUTO,
    E_MI_DIVP_FILTER_TYPE_BYPASS,
    E_MI_DIVP_FILTER_TYPE_BILINEAR,
    E_MI_DIVP_FILTER_TYPE_FILTER,
} MI_DIVP_FilterType_e;
```

- 成员

成员	描述
E_MI_DIVP_FILTER_TYPE_AUTO	Auto模式，会自动选择bypass或bilinear。
E_MI_DIVP_FILTER_TYPE_BYPASS	Bypass模式。
E_MI_DIVP_FILTER_TYPE_BILINEAR	Bilinear模式。
E_MI_DIVP_FILTER_TYPE_FILTER	Filter type数目。

- 注意事项
不做scaling默认是E_MI_DIVP_FILTER_TYPE_BYPASS filter type。
- 相关数据类型及接口
[MI_DIVP_StretchBufEx](#) [#214-mi_divp_stretchbufex]

3.5. MI_DIVP_OutputPortAttr_t

- 说明
定义 DIVP 绑定的output port的属性参数。
- 定义

```
typedef struct MI_DIVP_OutputPortAttr_s
{
    MI_U32 u32Width;//output width

    MI_U32 u32Height;//output height

    MI_SYS_PixelFormat_e ePixelFormat;

    MI_SYS_CompressMode_e eCompMode;//compress mode
}MI_DIVP_OutputPortAttr_t;
```

- 成员

成员名称	描述
u32Width	DIVP 通道输出画面的宽度
u32Height	DIVP 通道输出画面的高度
ePixelFormat	DIVP 通道输出画面的像素格式
eCompMode	DIVP 通道输出图像的压缩格式 · DIVP通道只能输出非压缩格式的图像

- 注意事项

各系列chip DIVP输出属性差异

芯片系列\输出属性	Output Pixel format	Output Stride alignment	Output Width alignment
MSR930	YUV422/YUV420(NV12)/ARGB8888/ABGR8888/ARGB1555/MST420	32	2
Pretzel	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Macaron	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Taiyaki/Takoyaki	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Pudding	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	

Ispahan	RGB565	32	2
	YUV422	32	
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Ikayaki	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	

3.6. MI_DIVP_ChnAttr_t

- 说明

定义 DIVP 通道的属性参数。

- 定义

```
typedef struct MI_DIVP_ChnAttr_s
{
    MI_U32 u32MaxWidth;//support max input width
    MI_U32 u32MaxHeight;//support max input height
    MI_DIVP_TnrLevel_e eTnrLevel;//TNR level
    MI_DIVP_DiType_e eDiType;//DI type
    MI_SYS_Rotate_e eRotateType;//rotate angle
```

```
MI_SYS_WindowRect_t stCropRect;//crop information

MI_BOOL bHorMirror;//horizontal mirror

MI_BOOL bVerMirror;//vertical mirror

}MI_DIVP_ChnAttr_t;
```

• 成员

成员名称	描述
u32MaxWidth	DIVP通道支援的input的最大宽度
u32MaxHeight	DIVP通道支援的input的最大高度
eTnrLevel	DIVP通道上TNR的等级
eDiType	DIVP通道上DI的类型
eRotateType	DIVP通道上画面旋转的角度
stCropRect	DIVP通道上的crop信息
bHorMirror	DIVP通道上水平方向翻转
bVerMirror	DIVP通道上垂直方向翻转

• 注意事项

芯片系列\输入属性	Input Pixel format	Input Stride alignment	Input Width alignment
MSR930	YUV422/YUV420(NV12)/ARGB8888/ABGR8888/ARGB1555/Tile Mode	32	YUV422:16 NV12:32
Pretzel	YUV422	32	2
	YUV420(NV12)	16	

	ARGB8888/ABGR8888	64	
	RGB565	32	
Macaron	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Taiyaki/Takoyaki	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Pudding	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Ispahan	YUV422	32	2
	YUV420(NV12)	16	
	ARGB8888/ABGR8888	64	
	RGB565	32	
Ikayaki	YUV422	32	16
	YUV420(NV12)	16	

3.7. MI_DIVP_DirectBuf_t

- 说明

定义用于图像处理的内存信息

- 定义

```
typedef struct MI_DIVP_DirectBuf_s
{
    MI_SYS_PixelFormat_e ePixelFormat; //YUV420SP or ARGB888
    only
    MI_U32 u32Width;
    MI_U32 u32Height;
    MI_U32 u32Stride[3];
    MI_PHY phyAddr[3];
}MI_DIVP_DirectBuf_t;
```

- 成员

成员名称	描述
ePixelFormat	图像的像素格式
u32Width	图像画面的宽度
u32Height	图像画面的高度
u32Stride	图像每行所占字节数
phyAddr	Buffer的起始物理地址

- 注意事项
 - 只支持YUV420SP或者ARGB8888的图像像素格式
 - u32Stride不能小于64
- 相关数据类型及接口

[MI_DIVP_StretchBuf](#) [#211-mi_divp_stretchbuf]

3.8. MI_DIVP_InitParam_t

- 说明

DIVP设备初始化参数
- 定义

```
typedef struct MI_DIVP_InitParam_s
{
    MI_U32 u32DevId;

    MI_U8 *u8Data;
} MI_DIVP_InitParam_t;
```

- 成员

成员名称	描述
u32DevId	设备ID
u8Data	数据指针buffer

- 相关数据类型及接口

[MI_DIVP_InitDev](#) [#212-mi_divp_initdev]

4. DIVP 错误码

表4-1 DIVP API 返回值

错误代码	宏定义	描述
0x0	MI_SUCCESS	succeeded
0xa00c2002	MI_DIVP_ERR_INVALID_CHNID	无效的channel ID
0xa00c2003	MI_DIVP_ERR_INVALID_PARAM	传入的参数无效
0xa00c2006	MI_DIVP_ERR_NULL_PTR	空指针异常
0xa00c201c	MI_DIVP_ERR_FAILED	DIVP的操作失败
0xa00c2005	MI_DIVP_ERR_NO_RESOUCE	无资源可以使用
0xa00c201c	MI_DIVP_ERR_NO_CONTENT	通道中无显示内容

5. PROCFS介绍

5.1. cat

- 调试信息

```
# cat /proc/mi_modules/mi_divp/mi_divp0
```

```
-----Common info for mi_divp-----
ChnNum  EnChnNum  PassNum  InPortNum  OutPortNum  CollectSize
16      1          1          1          1          0

-----CMDQ kickoff counter-----
DevId   current_buf_size  Peak_buf_size
0       0              0

each dev buf info:
offset          length          used_flag          task_name

-----Common info formi_divp only dump enabled chn-----
ChnId  PassNum  EnInPNum  EnOutPNum  MMAHeapName
0      0        1          1          (null)
ChnId   current_buf_size  Peak_buf_size  user_pid
0       0              4c000         609

each chn buf info:
offset          length          used_flag          task_name

-----Input port common info for mi_divp only dump enabled port-----
ChnId  PassId  PortId  user_buf_quota  UsrInjectQ_cnt  BindInQ_cnt  TotalPendingBuf_size  usrLockedInjectCnt
0      0      0          4              0              0              0              0
ChnId  PassId  PortId  newPulseQ_cnt  nextToDoPulseQ_cnt  curWorkingQ_cnt  workingTask_cnt  lazyRewindTask_cnt
0      0      0          0              0              0              0              0

ChnId  PassId  PortId  Enable  bind_module_id  bind_module_name  bind_ChnId  bind_PortId  bind_Type  bind_Param  enable

ChnId  PassId  PortId  SrcFrmrate  DstFrmrate  GetFrame/Ms  FPS  FinishCnt  RewindCnt
0      0      0        0/ 0        0/ 0        0/ 1  0.00      11612      0

-----Output port common info for mi_divp only for enabled port-----
ChnId  PassId  PortId  usrDepth  BufCntQuota  usrLockedCnt  totalOutPortInUsed  DrvBkRefFifoQ_cnt  DrvBkRefFifoQ_size
0      0      0          5          5              0              0              0              0
ChnId  PassId  PortId  UsrGetFifoQ_cnt  UsrGetFifoQ_size  UsrGetFifoQ_segnum  UsrGetFifoQ_discardnum
0      0      0          0              0              11612              0
ChnId  PassId  PortId  workingTask_cnt  finishedTask_cnt
0      0      0          0              0
ChnId  PassId  PortId  GetFrame/Ms  FPS  FinishCnt  RewindCnt  GetTotalCnt  GetOkCnt
0      0      0        10/1100    9.09  11612      0          11612      11612

-----BindPeerInputPortList-----
ChnId  PassId  PortId  Enable  bind_module_id  bind_module_name  bind_ChnId  bind_PortId  bind_Type  bind_Param  enable

----- Start dump DEV -----
ISRNum
11612
----- End dump DEV -----

----- start dump CHN info -----
ChnId  Status  AttrChg  FieldType0  FieldType1
0      2        0          0          0
16     1        0          0          0

ChnId  CropX  CropY  CropW  CropH  bHMirror  bVMirror  eDiType  eRotateType  eTnrLevel
ChnAttrPre  0      0      0      352    288      0        0        0        0        0
ChnAttrOrg  0      0      0      352    288      0        0        0        0        0
ChnAttrPre  16     0      0      0      0        0        0        0        0        0
ChnAttrOrg  16     0      0      0      0        0        0        0        0        0

----- end dump CHN info -----

----- start dump OUTPUT PORT info -----
ChnId  OutputChg  CompMode  PreWidth  PreHeight  Pixel  Stride  phyaddr  FinishTaskCnt  fps
0      0          0          352      288      11      352     3030a000  11612         10
16     0          0          0         0         0         0         0          0          0

----- end dump OUTPUT PORT info -----
```

- 调试信息分析

记录当前某DIVP device的使用状况，以及device属性、layer属性、inputport/outputport属性，可以动态地获取到这些信息，方便调试和测试。

- 参数说明

参数		描述
channelinfo	ChnId	0~16 16 Capture Channel 不对上层开放
	Status	0: INITED 1: CREATED 2: STARTED 3: STOPED 4: DISTROYED
	AttrChg	0 : not change 1 : changed
	FieldType0, FieldType1 (last two fields's type)	0: NONE //no field. 1: TOP //Top field only. 2: BOTTOM //Bottom field only. 3: BOTH //Both fields. 4: NUM
	ChnAttrPre	channel的上一次设置属性
	ChnAttrOrg	channel的原始设置属性
	CropX · CropY · CropW · CropH	上层设置的crop位置
	bHMirror bVMirror	水平翻转使能 垂直翻转使能 0:disable 1:enable
	eDiType	0: OFF, //off 1: 2D, ///2.5D DI 2: 3D, ///3D DI 3: NUM
	eRotateType	0: NONE, //Rotate 0 degrees 1: 90, //Rotate 90 degrees 2: 180, //Rotate 180 degrees 3: 270, //Rotate 270 degrees 4: NUM

	eTnrLevel	0: OFF, 1: LOW, 2: MIDDLE, 3: HIGH, 4: NUM
inputport info	ChnId	Channel ID 0~16
	InputChg	input port 属性是否发生变化
	bIPChg	0: 隔行和逐行之间没有发生变化 1: 隔行切换到逐行，or 逐行到隔行
	PreWidth	input port width
	PreHeight	input port height
	Pixel	E_MI_SYS_PIXEL_FRAME_YUV422_YUYV = 0, E_MI_SYS_PIXEL_FRAME_ARGB8888, E_MI_SYS_PIXEL_FRAME_ABGR8888, E_MI_SYS_PIXEL_FRAME_RGB565, E_MI_SYS_PIXEL_FRAME_ARGB1555, E_MI_SYS_PIXEL_FRAME_I2, E_MI_SYS_PIXEL_FRAME_I4, E_MI_SYS_PIXEL_FRAME_I8, E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_4 E_MI_SYS_PIXEL_FRAME_YUV_SEMIPLANAR_4 E_MI_SYS_PIXEL_FRAME_YUV_MST_420, //vdec mstar private video format E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE1_H: E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE2_H: E_MI_SYS_PIXEL_FRAME_YC420_MSTTILE3_H: E_MI_SYS_PIXEL_FRAME_FORMAT_MAX
	Stride	Input的stride
	2Pmode	b2P_Enable值，1表示enable，0表示disable
	EnqueueTask	Enqueue的task数目
	EnqueueTask	Enqueue的task数目

	GetInTaskCnt	OnPreProcessInput task的task数目
Outputport Info	ChnId	Channel ID 0~16
	OutputChg	Output 属性是否发生变化 0: not change 1: changed
	Pixel	同input pixel
	CompMode	0 //no compress 1 //compress unit is 256 bytes as a segment 2 //compress unit is the whole line 3 //compress unit is the whole frame 4 //number
	PreWidth PreHeight	上一次设置的output port 宽和高
	Stride	Output 的stride
	phyaddr	Output buffer 的物理地址
	FinishTaskCnt	Finish task的数目
	fps	Output port的帧率
Common Info	ChnNum	Channel的总数
	Pixel	同input pixel
	PassNum	Pass的总数
	InPortNum	Input port的个数
	OutPortNum	Output port的个数
	CollectSize	Attach的pstAllocator数目
	current_buf_size	当前申请的内存大小
	Peak_buf_size	使用内存的峰值

user_pid	进程id
user_buf_quota	User可以拿的buffer最大个数
UsrInjectQ_cnt	User inject buffer个数
BindInQ_cnt	前端给input port enqueue的buffer个数
TotalPendingBuf_size	当前input working队列中各个task总共所占的buffer size
usrLockedInjectCnt	用户拿到buffer的个数
newPulseQ_cnt	new_pulse_fifo_inputqueue的buffer个数
nextTodoPulseQ_cnt	next_todo_pulse_inputqueue的buffer个数
curWorkingQ_cnt	cur_working_input_queue的buffer个数
workingTask_cnt	input_working_tasklist的task个数
lazyRewindTask_cnt	需要retry的task个数
bind_module_id	绑定的模块id
bind_module_name	绑定的模块名字
bind_ChnId	绑定模块的channel id
bind_PortId	绑定模块的port id
bind_Type	绑定方式： E_MI_SYS_BIND_TYPE_FRAME_BASE = 0x00000001, E_MI_SYS_BIND_TYPE_SW_LOW_LATENCY = 0x00000002, E_MI_SYS_BIND_TYPE_REALTIME = 0x00000000 E_MI_SYS_BIND_TYPE_HW_AUTOSYNC = 0x00000008, E_MI_SYS_BIND_TYPE_HW_RING = 0x00000010
bind_Param	绑定的参数

SrcFrmrate	source 帧率
DstFrmrate	destination帧率
GetFrame/Ms	实际帧数/所用时间
FPS	实际帧率
RewindCnt	需要retry的task个数
usrDepth	User可以拿到output port buffer的最大个数
BufCntQuota	Output 可以申请buffer的最大个数
usrLockedCnt	User实际拿到的buffer个数
totalOutPortInUsed	Outport实际申请到的buffer个数
DrvBkRefFifoQ_cnt	for special driver which need to refer back pre-processed buffer
DrvBkRefFifoQ_size	stDrvBkRefFifoQueue 所占用的buffer size总数
UsrGetFifoQ_cnt	Duplicate 到output port的buffer个数
UsrGetFifoQ_size	Duplicate 到output port的buffer所占的总大小
UsrGetFifoQ_seqnum	output拿到buffer的总数
UsrGetFifoQ_discardnum	Output由于申请新的buffer需求而遗弃原来buffer个数
finishedTask_cnt	Output finish task的个数
GetTotalCnt	Output拿到buffer总个数
GetOkCnt	统计index为E_MI_SYS_FRC_OBTAIN的帧数
ISRNum	收到的中断总数

5.2. echo

功能	打印接收到每一帧的PTS，如果前端是VDEC，会打印VDEC送过来的FrameID， Debug情况下FrameID和PTS的值都是递增的。
命令	echo checkframeid [ChnID] [Status] > /proc/mi_modules/mi_divp/mi_divp0
参数说明	[ChnID] 通道号 [0~32] [Status] ON 开始打印 OFF 停止打印
举例	<pre>echo checkframeid 0 ON/OFF > /proc/mi_modules/mi_divp/mi_divp0</pre> [MI DIVP PROCFS]:ChnID 0, receive buffer ID = 63 , H = 0, L = 196615, PTS = 439560([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 64 , H = 0, L = 196616, PTS = 439890([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 65 , H = 0, L = 196615, PTS = 440220([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 66 , H = 0, L = 196615, PTS = 440550([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 67 , H = 0, L = 196615, PTS = 440880([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 68 , H = 0, L = 196615, PTS = 441210([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 69 , H = 0, L = 196614, PTS = 441540([MI DIVP PROCFS]:ChnID 0, receive buffer ID = 70 , H = 0, L = 196615, PTS = 441870(receive buffer ID：每个channel接收到buffer的ID。 H/L: VDEC 送来的FrameID，H高32位，L低32位，前端不是VDEC则无该打印。PTS: 的PTS。

功能	统计1秒时间内GetBuffer的数量，以及每拿到两个inputbuffer之间的间隔时间的 平均最小值，Debug丢帧问题。
命令	echo statintervaltime [ChnID] ON/OFF > /proc/mi_modules/mi_divp/mi_divp0
参数说明	[ChnID] 通道号 [0~32] [Status] ON 开始统计并打印 OFF 停止统计和打印
举例	<pre>echo statintervaltime 0 ON > /proc/mi_modules/mi_divp/mi_divp0</pre> [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 50393 us,MAX 59601 us,MIN 41401 us,57534 us,42415 us,42896 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 50247 us,MAX 63004 us,MIN 36896 us,62845 us,37120 us,37130 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:18,Average time: 51049 us,MAX 70431 us,MIN 38873 us,59736 us,40109 us,40283 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:20,Average time: 48537 us,MAX 67168 us,MIN 18513 us,66953 us,31242 us,32681 us [MI DIVP PROCFS]:ChnIID 0,1s FrameCnt:19,Average time: 49962 us,MAX 61416 us,MIN 38564 us,61338 us,38736 us,38762 us FrameCnt: 1秒时间GetBuffer的数量 Average time：拿到两个buffer之间间隔时间的平均时间 MAX：间隔时间最大的三个值。 MIN: 间隔时间最小的三个值



功能	DIVP不再处理某一个channel，以节省频宽，验证频宽相关问题
命令	echo stoponechannel [ChnID] [Status] > /proc/mi_modules/mi_divp/mi_divp0
参数说明	[ChnID] 通道号 [0~32] [Status] ON 停止处理设置的channel OFF继续处理设置的channel
举例	<pre>echo stoponechannel 0 ON > /proc/mi_modules/mi_divp/mi_divp0</pre> 通道0的画面将会停住



功能	Dump一张input frame。
命令	<code>echo dumpframe [ChnID] [frameStorePath] > /proc/mi_modules/mi_divp/mi_divp0</code>
参数说明	[ChnID] 通道号 [0~16] [frameStorePath] 指定保存input frame的位置。
举例	<code>#echo dumpframe 0 /customer > /proc/mi_modules/mi_divp/mi_divp0</code> Dump 一张 input frame保存在/customer这个路径下。 默认input frame名字比如：dump_divp_chn0_frame_w_352_h_288.yuv



功能	打开mi divp的debug信息。
命令	<code>echo debug 0/1 > /proc/mi_modules/mi_divp/mi_divp0</code>
参数说明	[0/1] 0表示关闭，1表示打开。
举例	<code>#echo debug 1 > /proc/mi_modules/mi_divp/mi_divp0</code> <code>printk in _MI_SYS_IMPL_Common_WriteProc</code> <code>divp debug:on</code> <code>#echo debug 0 > /proc/mi_modules/mi_divp/mi_divp0</code> <code>printk in _MI_SYS_IMPL_Common_WriteProc</code> <code>divp debug:off</code>

